

Git, GitHub, and Codespaces

What we will build—and what you will be able to do

The finished repository:

```
growth-calculator/  
  README.md  
  growth.py  
  python_basics.py
```

By the end, you can

- create a repository and a Codespace;
- edit and run Python code;
- inspect, stage, and commit changes;
- push work to GitHub and pull new work;
- run and change code you did not write.

We will learn Git by repeatedly using it on one small project.

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

Why not save a new file every time?

growth.py
growth_final.py
growth_final_v2.py
growth_final_v2_fixed.py
growth_REALLY_FINAL.py

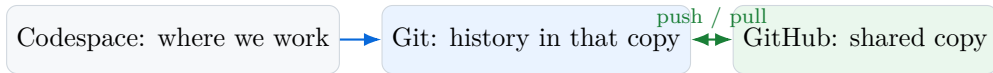
This does not tell us:

- what changed between versions;
- why a change was made;
- which code generated a result;
- whether two collaborators changed the same file.

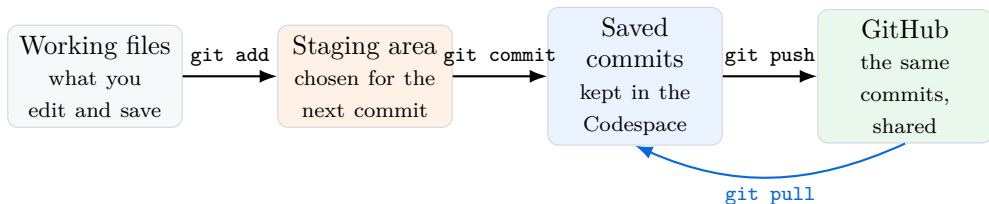
Git stores named snapshots of the project and a history connecting them.

Three tools, three different jobs

- Git records versions of files and connects them in a history.
- GitHub hosts a repository online so it can be shared and synchronized.
- Codespaces provides a cloud computer and VS Code editor in the browser.



The Git pipeline: where is a change?



Saving a file, committing it, and pushing it are three separate actions. Keeping them apart prevents most of the confusion that follows.

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

Step 1: open GitHub's repository form

In the GitHub website:

1. Sign in at `github.com`.
2. In the upper-right corner, click .
3. Select .

A repository is the project folder plus the Git history attached to it.

Step 2: configure and create the repository

Fill in the form as follows.

Owner	<code>your-username</code>
Repository name	<code>growth-calculator</code>
Description	A Python growth calculator
Visibility	Private
Add a README file	yes

Then click .

Checkpoint: the repository exists on GitHub.

We add a README because

- it explains the project to a visitor;
- it gives the repository a first file;
- GitHub creates the first commit for us.

Step 3: start a Codespace

From the repository page:

1. Click the green **Code** button.
2. Select the **Codespaces** tab.
3. Click **Create codespace on main**.

GitHub opens a new browser tab.

Starting the cloud computer takes a minute or two; wait for it rather than reloading the page.

The Codespace contains a working copy of the GitHub repository.

Orient yourself inside Codespaces



We will use three areas:

- the Explorer, to create and open files;
- the editor, to write code;
- the terminal, to run commands.

Open the terminal with

Terminal →

New Terminal .

Step 4: check that Git knows who you are

Every commit is signed with a name and an email. Ask what this machine will use:

```
$ git config --global user.name  
$ git config --global user.email
```

If both print something, you are done. If either prints nothing, set it, using your own name and the email you signed up to GitHub with:

```
$ git config --global user.name "Your Name"  
$ git config --global user.email "you@example.com"
```

This is set per machine, not per repository, so a new Codespace may ask again. Commits made with the wrong email still work; they are just not credited to your GitHub account.

Inspect the starting repository

Type in the terminal:

```
$ pwd
/workspaces/growth-calculator
$ ls
README.md
$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Read the output as follows:

- **pwd:** show the current folder.
- **ls:** list the files in it.
- **main:** the current branch.
- **origin/main:** GitHub's corresponding branch.
- **clean:** no unsaved Git changes.

Checkpoint: both copies are synchronized.

Read the existing history

```
$ git log --oneline  
a1b2c3d (HEAD -> main, origin/main)  
    Initial commit
```

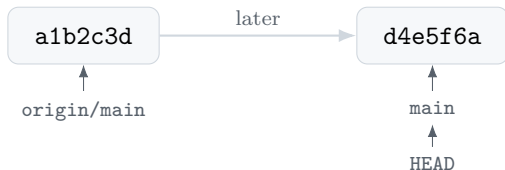
The output says that

- `a1b2c3d` is the shortened commit identifier;
- `HEAD -> main` is where our Codespace currently points;
- `origin/main` is where GitHub stood the last time we asked;
- `Initial commit` is the saved description.

Both labels point to the same commit, so the two copies are synchronized.

Three labels: `main`, `origin/main`, `HEAD`

Commits are the only things Git stores. Each one has an identifier, and each one stays exactly as it was written. Everything else is a label that points at a commit, and labels move.



- `main` is our branch. It moves forward every time we commit.
- `origin/main` records where GitHub's `main` stood the last time we asked GitHub. It moves only when we run `fetch`, `pull`, or `push`.
- `HEAD` is where we are working. `HEAD -> main` means the next commit will move `main`.

Reading the state from the two labels

Every question about “where is my work?” is answered by the position of `main` against `origin/main`.

In <code>git log -oneline</code>	What it means	What to do
Both labels on the same commit <code>main</code> above <code>origin/main</code>	The two copies agree We have work GitHub does not	Nothing <code>git push</code>
<code>origin/main</code> above <code>main</code>	GitHub has work we do not	<code>git pull</code>
Both moved since they last agreed	Each side has new work	<code>git pull</code> , then <code>git push</code>

`git status` reports the same thing in words. It answers from what we learned the last time we asked, because Git never contacts GitHub on its own. Run `git fetch` first and the answer describes GitHub as it is now.

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

Create the program: growth.py

In the Explorer of your
Codespace:

1. Click the **New File** icon.
2. Name the file `growth.py`.
3. Enter the code shown here.
4. Save with **Ctrl+S**.

No package installation is
needed.

```
def percentage_change(old_value, new_value):  
    """Return percentage change from old to new."""  
    if old_value == 0:  
        raise ValueError("old_value cannot be zero")  
  
    return 100 * (new_value - old_value) / old_value  
  
if __name__ == "__main__":  
    result = percentage_change(100, 105)  
    print(f"Percentage change: {result}%")
```

Run the program before saving a Git version

```
$ python growth.py  
Percentage change: 5.0%
```

Try a different pair of values by changing:

```
result = percentage_change(100, 105)
```

Python executed the file and the program produced the expected result. Git has not saved a new version, and GitHub has not received anything.

Running code and recording its version are separate actions.

Ask Git what changed

```
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what
   will be committed)
    growth.py

nothing added to commit
```

“Untracked” means that the file exists in the Codespace, that Git sees it, and that it has never been included in a commit.

Note that `git diff` shows changes to tracked files. It will not display the contents of this new, untracked file.

Stage the file, then review exactly what is staged

```
$ git add growth.py
$ git status
Changes to be committed:
  new file:   growth.py

$ git diff --staged
diff --git a/growth.py b/growth.py
new file mode 100644
--- /dev/null
+++ b/growth.py
@@ ...
+def percentage_change(old_value, new_value):
```

`git add` selected `growth.py` for the next commit, but it did not create the commit. `git diff --staged` previews the exact snapshot.

State: staged, not committed.

If output is longer than the terminal, Git shows it in a *pager* and the prompt disappears. Nothing has crashed: `Space` scrolls, and `q` returns you to the prompt.

Commit: save one coherent change

```
$ git commit -m "Add growth calculator"
[main d4e5f6a] Add growth calculator
 1 file changed, 11 insertions(+)
 create mode 100644 growth.py

$ git log --oneline
d4e5f6a (HEAD -> main) Add growth calculator
a1b2c3d (origin/main) Initial commit
```

The commit identifier is d4e5f6a. HEAD -> main moved to the new commit; origin/main did not move. The commit therefore exists locally, but not yet on GitHub.

State: committed locally.

Push: send local commits to GitHub

```
$ git push
Enumerating objects: 4, done.
Writing objects: 100% (3/3), done.
To https://github.com/your-username/
growth-calculator.git
   a1b2c3d..d4e5f6a  main -> main
```

Then return to the GitHub tab and refresh the repository page.

Checkpoint: Codespace and GitHub contain the new commit.

Verify on GitHub that

1. `growth.py` is visible;
2. the commit message is visible;
3. the history shows two commits.

A successful local commit does not imply a successful push.

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

Simulate a collaborator by editing on GitHub

On the GitHub repository page:

1. Click `README.md`.
2. Click the pencil icon `Edit this file`.
3. Add the text shown at right.
4. Click `Commit changes...`.
5. Enter Describe the project and confirm.

The new content of `README.md`:

```
# Growth calculator  
A small Python program that  
calculates percentage changes.
```

GitHub is now one commit ahead of the Codespace.

Fetch, inspect, then pull the new commit

```
$ git fetch
From https://github.com/.../growth-calculator
 6141a87..5f89e4d  main -> origin/main

$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit,
and can be fast-forwarded.
  (use "git pull" to update your local branch)

$ git pull
Updating 6141a87..5f89e4d
Fast-forward
 README.md | 3 +++
 1 file changed, 3 insertions(+)
```

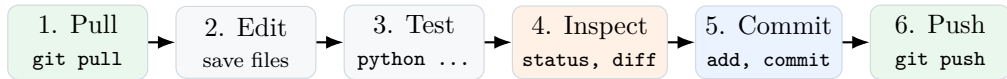
1. `git fetch` contacts GitHub and refreshes `origin/main`; it does not change your files.
2. `git status` now sees that GitHub is one commit ahead.
3. `git pull` moves `main` forward and updates `README.md`.

“Fast-forward” means there was no competing local commit to merge.

Outline

1. Why Git
2. Setup
3. Add, commit, push
4. Pulling
5. Summary

The daily workflow



Before the commit, ask

- does the program run?
- do the tests pass?
- did I inspect every changed line?

After the commit, ask

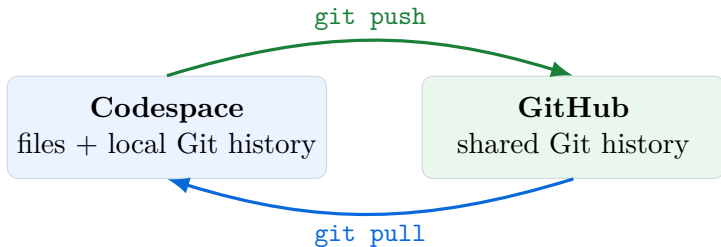
- is the message meaningful?
- did the push succeed?
- can I see the commit on GitHub?

Command guide: question first, command second

Question	Command
What is the repository's current state?	<code>git status</code>
What unstaged tracked lines changed?	<code>git diff</code>
What exactly is selected for the next commit?	<code>git diff -staged</code>
Which changes should enter the next commit?	<code>git add <file></code>
How do I save the staged snapshot?	<code>git commit -m "message"</code>
What commits exist in the history?	<code>git log -oneline</code>
How do I receive GitHub's newer commits?	<code>git pull</code>
How do I send my local commits to GitHub?	<code>git push</code>

When uncertain, start with `git status`; read Git's message before acting.

The one picture to remember



Pull → Edit → Test → Inspect → Commit → Push

Use it on every project until the workflow becomes routine.

Now: somebody else's code

Everything so far has been yours. You wrote it, so you already knew what it was supposed to do.

The rest of the session begins from the opposite position. Your repository contains a file you did not write.

That is the ordinary situation on any project, and the task it sets is always the same:

1. run it;
2. follow what each part does;
3. change one thing;
4. record the change and push it.

We move to the Codespace and work through `python_basics.py` together.

Set it up

```
$ curl -O https://raw.githubusercontent.com/ChristosMylonakisCEMFI/CSS-DataScience/main/slides/python-basics/python_basics.py
$ git add python_basics.py
$ git commit -m "Add the Python basics file"
$ pip install pandas matplotlib
```

Then open the **Extensions** panel on the left bar and install **Python**, by Microsoft.

Commit it before you change it. `git diff` can only show changes to a file Git already knows about.

Run it

Select a few lines. Press `Shift+Enter`.

They run in the terminal below, and the output appears there.

Work down the file a few lines at a time. We will do it together.

Change one thing

Once we have been through the file, make one small edit of your own.

In section 6, the histogram is drawn with 40 bars. Halve that, and run the block again.

The shape of the distribution should survive. The detail should coarsen.

Keep it small and deliberate. The test is whether you can describe what you changed in one short sentence — because in a moment you will have to.

Record and publish your change

```
$ git status
$ git diff
$ git add python_basics.py
$ git commit -m "Use 20 histogram bars"
$ git push
```

The same five commands as this morning, on somebody else's file instead of your own:

- **status:** what has changed;
- **diff:** exactly which lines;
- **add:** choose what to record;
- **commit:** record it, with a description;
- **push:** send it to GitHub.

`git diff` shows your one line and nothing else. Refresh your repository page on GitHub and the change is there, with your name against it.

What you have just practised

Every idea below is one you will meet on a real project. You have now met all of them once.

The idea	Where you met it
Put your own work under version control	<code>growth.py</code> : add, commit, push
Someone else changed the shared copy	the edit made on GitHub, then <code>fetch</code> and <code>pull</code>
Code arrives that you did not write	fetching <code>python_basics.py</code>
Understand it well enough to change it	its section 7
Record a change so others can read it	<code>git diff</code> , then a commit message that says what you did
Publish it	<code>git push</code>
Two edits collide	merge conflicts

Together: the workflow used *inside* a project.

Appendix

Create package-free tests: `test_growth.py`

Create a new file named `test_growth.py`.

Each `assert` states a result that must be true. If one is false, Python stops with an error.

`growth` refers to our own `growth.py` file. The tests use only Python itself.

```
from growth import percentage_change

assert percentage_change(100, 105) == 5
assert percentage_change(100, 90) == -10
assert percentage_change(100, 100) == 0

print("All tests passed.")
```

Run the tests—and see what failure looks like

When all expectations are correct:

```
$ python test_growth.py  
All tests passed.
```

Temporarily change the first expected result from 5 to 6 and rerun.

A failing test:

```
$ python test_growth.py  
Traceback (most recent call last):  
  File "test_growth.py", line 4  
    assert percentage_change(100,105) == 6  
AssertionError
```

Restore 5 and rerun until all tests pass.

A test converts an expected result into a check the computer can repeat.

Record the tested change

```
$ git status
$ git diff
$ python test_growth.py
All tests passed.
$ git add test_growth.py
$ git diff --staged
$ git commit -m "Add tests for growth calculator"
$ git push
```

The order matters:

1. identify changed files;
2. inspect their contents;
3. verify the code works;
4. select the intended change;
5. review, commit, and share it.

A good commit is small, coherent, reviewed, and tested.

What is .gitignore?

`.gitignore` is a plain-text file that tells Git which files or folders it should not track. Each line names a path or pattern to ignore; that line is sometimes called a “rule.”

Nothing is deleted. The directory stays on the computer; Git simply stops offering to include it in commits.

For this project, the complete file is one line:

```
__pycache__/
```

It means: “Git, ignore the `__pycache__/` directory that Python created when the tests imported `growth.py`.”

Create, check, and commit .gitignore

Create the file:

```
$ printf "__pycache__/\n" > .gitignore
$ cat .gitignore
__pycache__/
```

`printf` writes the line into `.gitignore`;
`cat` checks what was written.

Check the result:

```
$ git status --short
?? .gitignore
```

The cache is no longer listed because Git is ignoring it.

Share the instruction:

```
$ git add .gitignore
$ git commit -m "Ignore Python cache"
$ git push
```

We commit `.gitignore` so everyone using the repository gets the same instruction: `.gitignore` is tracked, `__pycache__/_` is ignored.

If a push is rejected, synchronize first

```
$ git push
! [rejected] main -> main (fetch first)
error: failed to push some refs
```

This means GitHub has commits that your Codespace does not have. Git refuses to overwrite them.

Do not solve this with force: for this course workflow, never use `git push -force`, which can discard shared history.

The safe response is

```
$ git status
$ git pull
# Read any message from Git carefully.
$ python test_growth.py
$ git push
```

If Git reports a conflict, stop and resolve the marked files before pushing.

What is a merge conflict?

A conflict occurs when Git cannot decide how to combine competing edits automatically. Git marks the disputed region:

```
«««< HEAD
Percentage change calculator
=====
Growth rate calculator
»»»> origin/main
```

A conflict is a request for human judgment, not evidence that work was lost.

To resolve it,

1. read both versions;
2. edit the file to the intended final content;
3. remove all conflict markers;
4. run the program and tests;
5. stage and commit the resolution.